



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 2

Experiment Title: Implementation of Shortest Job First (SJF)
CPU Scheduling Algorithm.

Date of Submission : 09-08-2026

Submitted By:

Name : Boishakhi Basu.

Student Id : 22201173

Semester : 4th Year 1st Semester

Section : C-2

Submitted To
Aita Rahman Orthi
Lecturer , UAP

Problem Statement:

Given the following processes with their Arrival Time (AT) and Burst Time (BT):

Process	AT	BT
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

Implement **SJF (Non-Preemptive)** and **FCFS** scheduling algorithms. Calculate **CT, TAT, WT, Average TAT, and Average WT**, represent the execution sequence, and compare both algorithms to determine which performs better.

Source Code:

```
# SJF Non - Preemptive
```

```
process = ["p1", "p2", "p3", "p4", "p5"]
```

```
at = [3, 2, 5, 1, 6]
```

```
bt = [3, 5, 4, 3, 2]
```

```
n = len(process)
```

```
ct = [0] * n
```

```
wt = [0] * n
```

```
done = [0] * n
```

```
tat = [0] * n
```

```
time = 0
```

```
completed = 0
```

```
while completed < n:
```

```
    x = -1
```

```

for i in range(n):

    if at[i] <= time and done[i] == 0:

        if x == -1 or bt[i] < bt[x]:
            x = i

    if x == -1:
        time += 1

    else:
        time = time + bt[x]

        ct[x] = time
        done[x] = 1
        completed += 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage TAT =", sum(tat)/n)
print("Average WT =", sum(wt)/n)

print("\n")

# FCFS
processes = [
    ["P1", 3, 3],
    ["P2", 2, 5],

```

```
["P3", 5, 5],
["P4", 1, 3],
["P5", 6, 2]
]

processes.sort(key=lambda x: x[1])

time = 0
total_tat = 0
total_wt = 0

print("PID\tAT\tBT\tCT\tAT\tWT")

for pid, at, bt in processes:
    if time < at:
        time = at

    time = time + bt
    ct = time

    tat = ct - at

    wt = tat - bt

    total_tat += tat
    total_wt += wt

    print(pid, "\t", at, "\t", bt, "\t", ct, "\t", tat, "\t", wt)

ava_tat = total_tat / len(processes)
ava_wt = total_wt / len(processes)

print("\nAverage TAT =", ava_tat)
print("Average WT =", ava_wt)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Average WT = 5.2
PS C:\Users\lab\Desktop\New folder> & C:\Users\lab\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\lab\AppData\Local\Programs\Python\Python38-64\Scripts\python.exe /New folder/SJF.py"

Process AT      BT      CT      TAT      WT
p1      3      3      7      4      1
p2      2      5      18     16     11
p3      5      4      13      8      4
p4      1      3      4      3      0
p5      6      2      9      3      1

Average TAT = 6.8
Average WT = 3.4

PID      AT      BT      CT      AT      WT
P4      1      3      4      3      0
P2      2      5      9      7      2
P1      3      3      12     9      6
P3      5      5      17     12     7
P5      6      2      19     13     11

Average TAT = 8.8
Average WT = 5.2
PS C:\Users\lab\Desktop\New folder> 
```

Discussion:

In this experiment, **Non-Preemptive SJF and FCFS CPU scheduling algorithms** were implemented and compared. The processes were scheduled according to their Arrival Time and Burst Time, and **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)** were calculated. SJF selects the available process with the shortest burst time, while FCFS executes processes according to their arrival order. The average TAT and WT were then calculated to compare the performance of both algorithms. It was observed that the scheduling order and performance metrics differ because the two algorithms use different process-selection strategies

Conclusion:

The experiment successfully implemented **Non-Preemptive SJF and FCFS scheduling algorithms** and compared their performance. SJF generally provides lower average waiting and turnaround time by executing shorter jobs first, while FCFS is simpler and follows the order of process arrival. For the given processes, the calculated **Average TAT and Average WT** can be used to determine the better-performing algorithm. Overall, the experiment helped demonstrate how different CPU scheduling strategies affect system performance.

GitHub:

<https://github.com/boishakhi73/CSE-402-Operating-System-Lab>